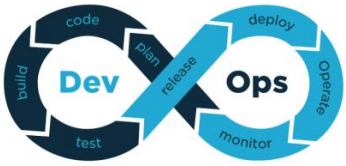
FATIMA JINNAH WOMEN UNIVERSITY
RAWALPINDI

Cloud DevOps



Lab 05

Build, Test, and Package Applications with IntelliJ IDEA and Docker

Submitted by: Rabbia Noor

Registration No.: 2023-BCS-070

Course Code: CS-649

GitHub Username: Rabbia-Noor

Section: A

DevOps Repository: https://github.com/Rabbia-Noor/DevOps

Table of Contents

Task 1 – Clone and Launch IntelliJ IDEA 3

Task 2 – Build and Run the Maven Project 4

Task 3 – Build and Run the Gradle Project 8

Task 4 – Install and Run the Node.js Project 11

Task 5 – Build the React and Node.js API Artifact 14

Task 6 – Build a Docker Image for the Node.js Project 16

Build and Run Results 18

Exam Evaluation Questions 18

DevOps repository (Labs/Lab05): 19

Task 1 – Clone and Launch IntelliJ IDEA

Step 1: Cloned the supplied IntelliJ Docker repository on the Ubuntu host with git clone https://github.com/WaqasSaleem97/Intellij-Idea-docker.git. The clone finished (35/35 objects), and ls inside Intellij-Idea-docker shows setup.sh, compose.yaml and Dockerfile. The prompt shows rabbia-noor@ubuntu.

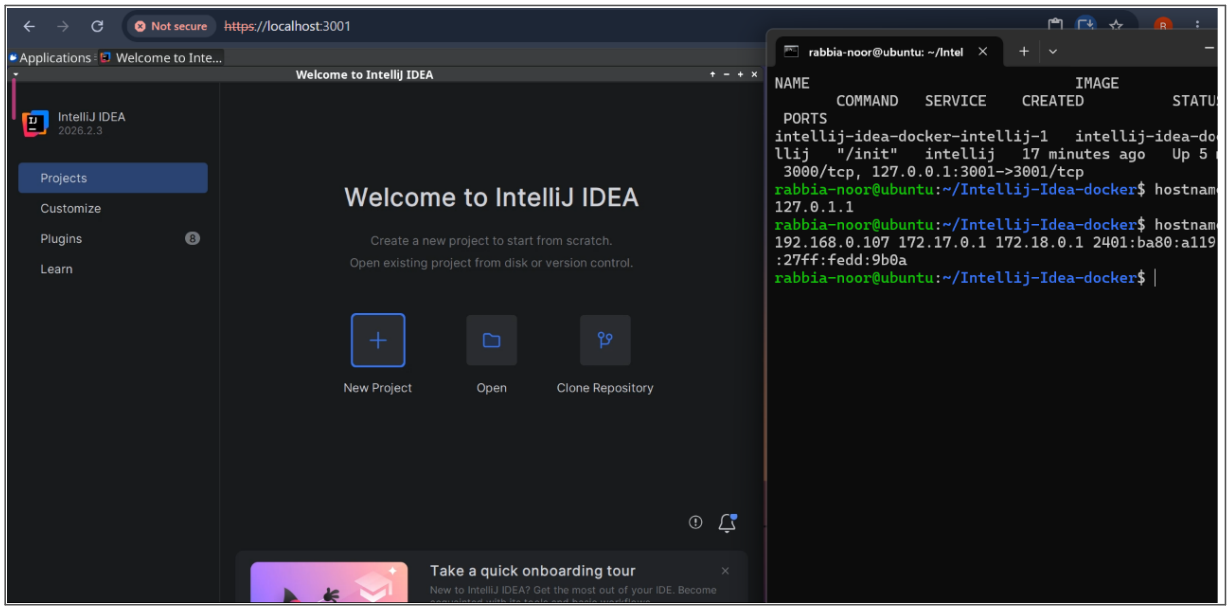
```
rabbia-noor@ubuntu:~$ git clone https://github.com/WaqasSaleem97/Intellij-Idea-docker.git
Cloning into 'Intellij-Idea-docker'...
remote: Enumerating objects: 35, done.
remote: Counting objects: 100% (35/35), done.
remote: Compressing objects: 100% (32/32), done.
remote: Total 35 (delta 16), reused 5 (delta 0), pack-reused 0 (from 0)
Receiving objects: 100% (35/35), 15.24 KiB | 11.00 KiB/s, done.
Resolving deltas: 100% (16/16), done.
rabbia-noor@ubuntu:~$ cd Intellij-Idea-docker
rabbia-noor@ubuntu:~/Intellij-Idea-docker$ ls
Dockerfile  README.md  compose.yaml  init-terminal.sh  intellij-idea.desktop  remove.sh  resources.sh  setup.sh
rabbia-noor@ubuntu:~/Intellij-Idea-docker$
```

Step 2: Built and started the environment with bash setup.sh. The image intellij-idea-docker-intellij was built and tagged, and docker compose ps shows the intellij service Up, with desktop port 127.0.0.1:3001->3001/tcp.

```
Successfully built 40ac0d4e8879
Successfully tagged intellij-idea-docker-intellij:latest
[+] build 1/1
 ✔Image intellij-idea-docker-intellij Built                                                    33.1s
[+] up 1/1
 ✔Container intellij-idea-docker-intellij-1 Running                                             0.0s
NAME                          IMAGE                         COMMAND   SERVICE    CREATED          STATUS          PORTS
intellij-idea-docker-intellij-1   intellij-idea-docker-intellij   "/init"   intellij   17 minutes ago   Up 5 minutes   3000/tcp, 127.
0.0.1:3001->3001/tcp

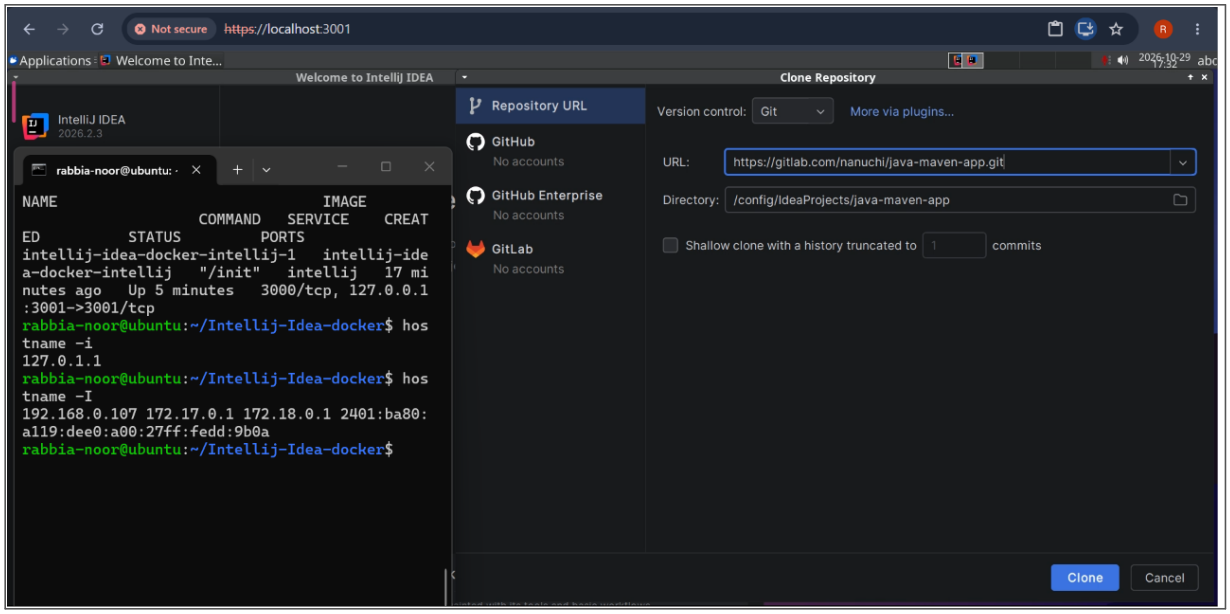
Browser username: admin (or WEBTOP_USER from your existing .env).
Open https://localhost:3001 in your host computer browser.
rabbia-noor@ubuntu:~/Intellij-Idea-docker$ docker compose ps
NAME                          IMAGE                         COMMAND   SERVICE    CREATED          STATUS          PORTS
intellij-idea-docker-intellij-1   intellij-idea-docker-intellij   "/init"   intellij   17 minutes ago   Up 5 minutes   3000/tcp, 127.
0.0.1:3001->3001/tcp
rabbia-noor@ubuntu:~/Intellij-Idea-docker$
```

Step 3: Opened https://localhost:3001 in the host browser and signed in. IntelliJ IDEA 2026.2.3 shows its Welcome screen in the browser desktop, with the Ubuntu host terminal (rabbia-noor@ubuntu) beside it.

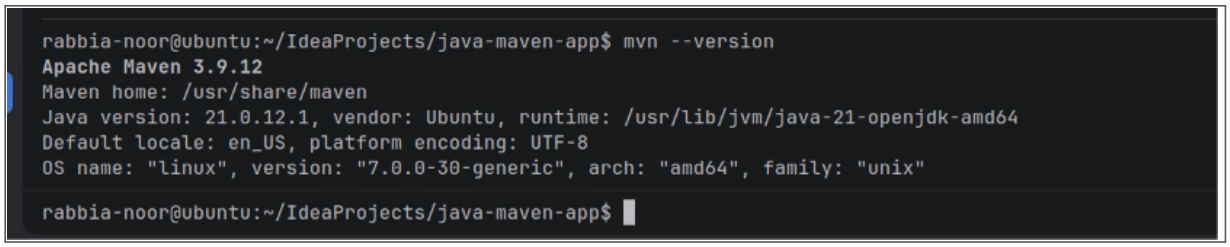


Task 2 – Build and Run the Maven Project

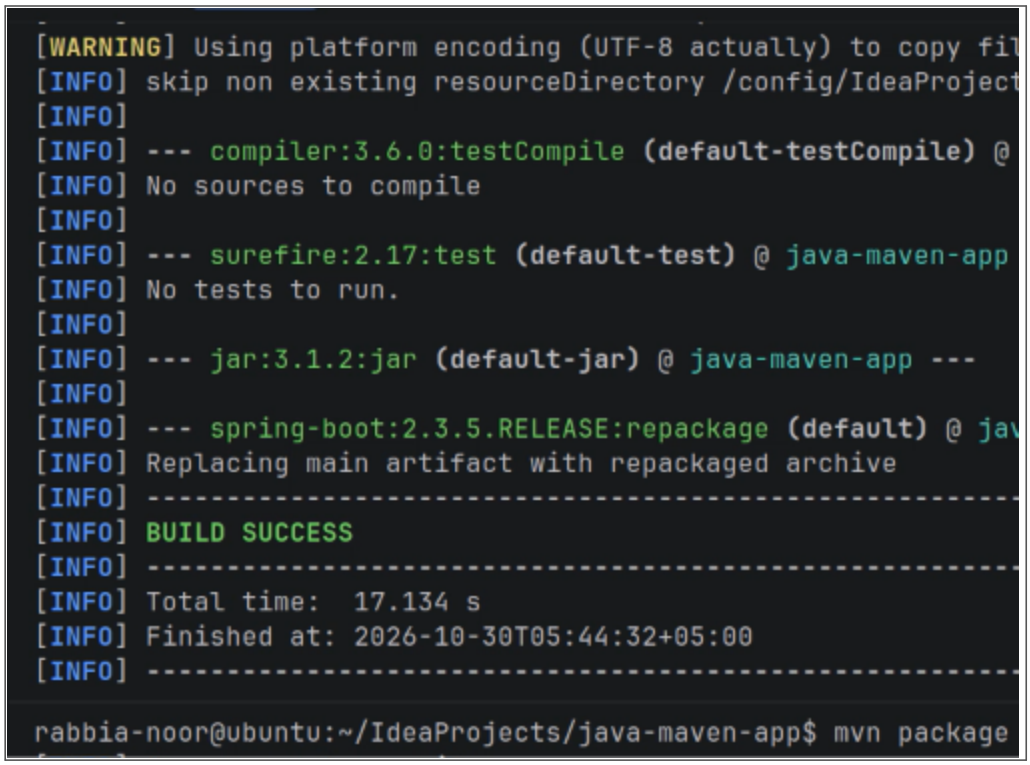
Step 1: On the Welcome screen, Clone Repository was used with URL https://gitlab.com/nanuchi/java-maven-app.git and directory /config/IdeaProjects/java-maven-app. The host terminal is visible beside the dialog.



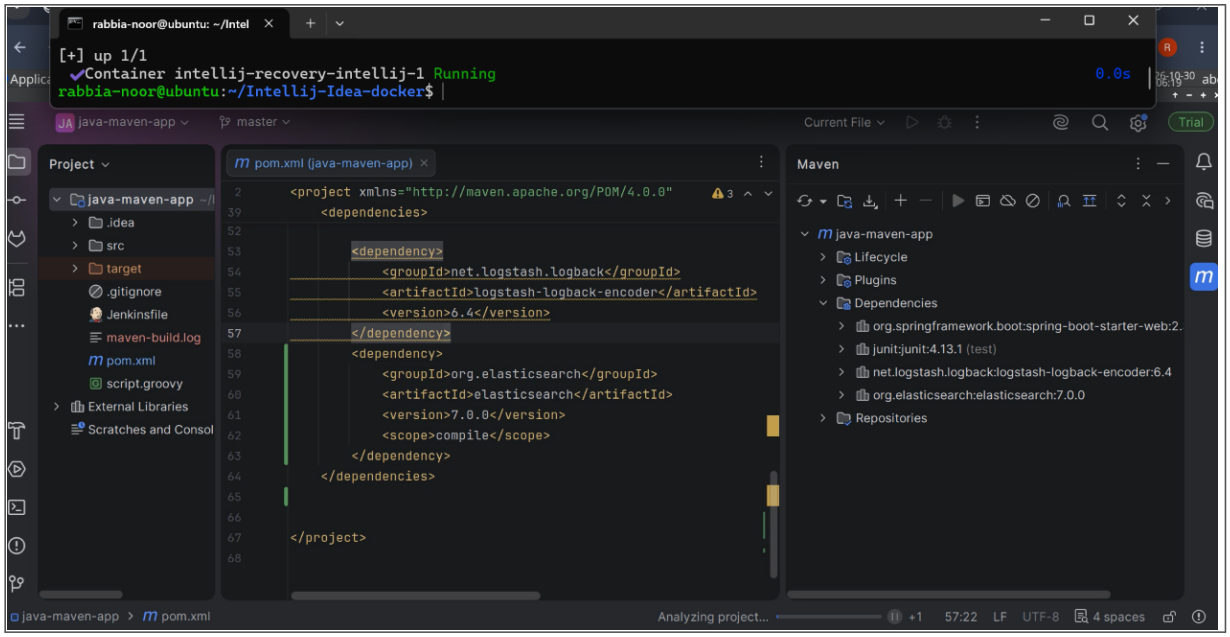
Step 2: Installed Maven with sudo apt install -y maven. In IntelliJ's terminal, mvn --version reports Apache Maven 3.9.12 with Java 21.0.12.1 (Ubuntu OpenJDK).



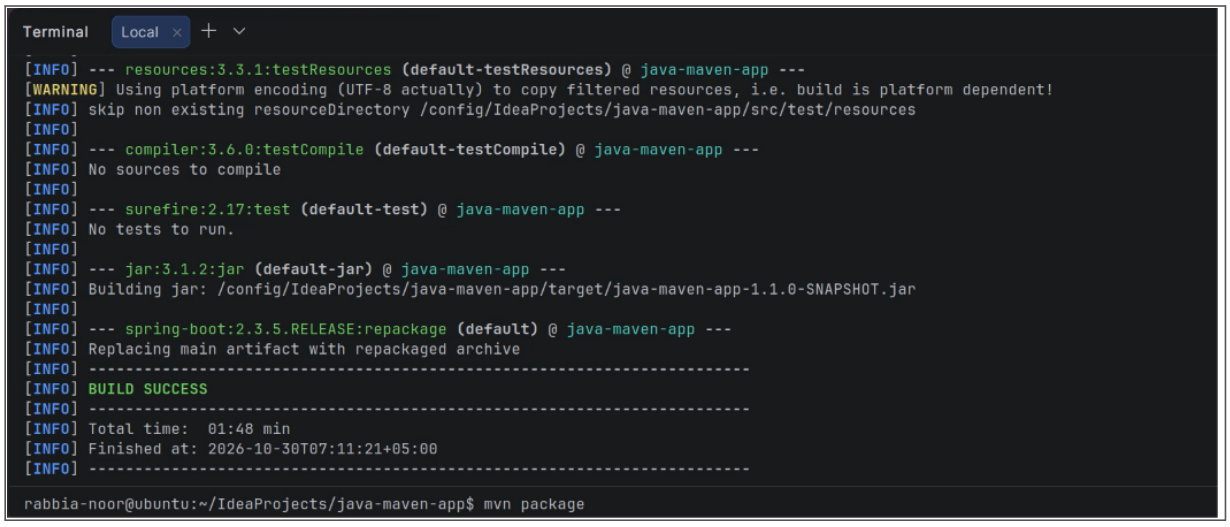
Step 3: Ran mvn package in the project root. The build ended with BUILD SUCCESS (total time 17.134 s) after the jar and spring-boot repackage steps.



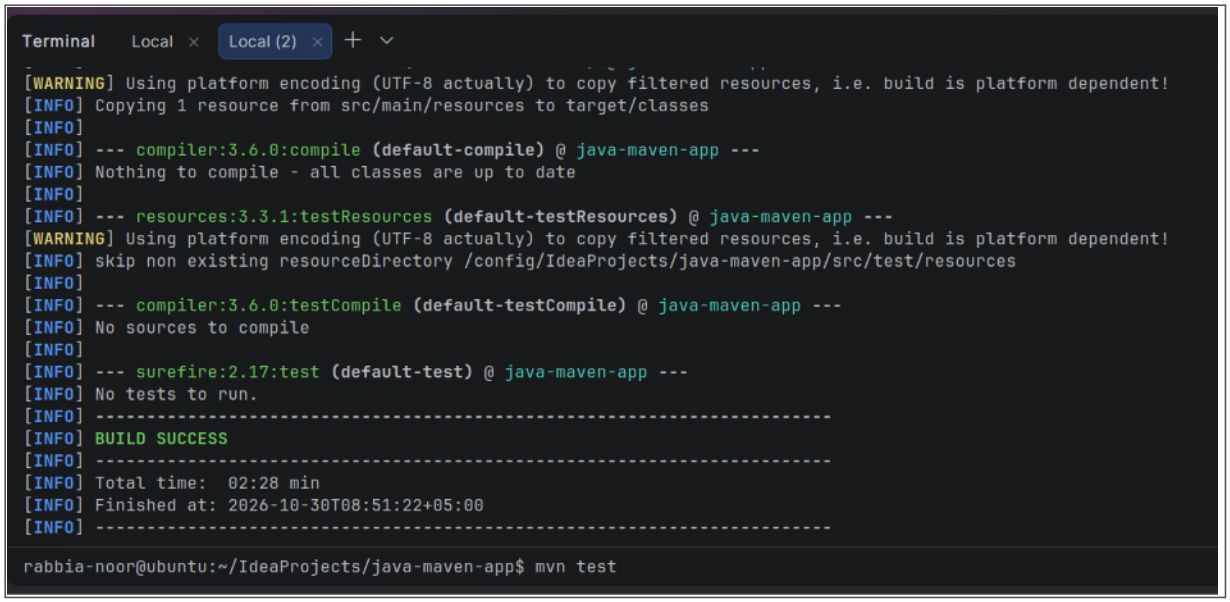
Step 4: Added the org.elasticsearch:elasticsearch:7.0.0 dependency (scope compile) to pom.xml and reloaded the Maven project. The Maven tool window now lists org.elasticsearch:elasticsearch:7.0.0 under Dependencies. The host terminal prompt rabbia-noor@ubuntu is visible at the top.



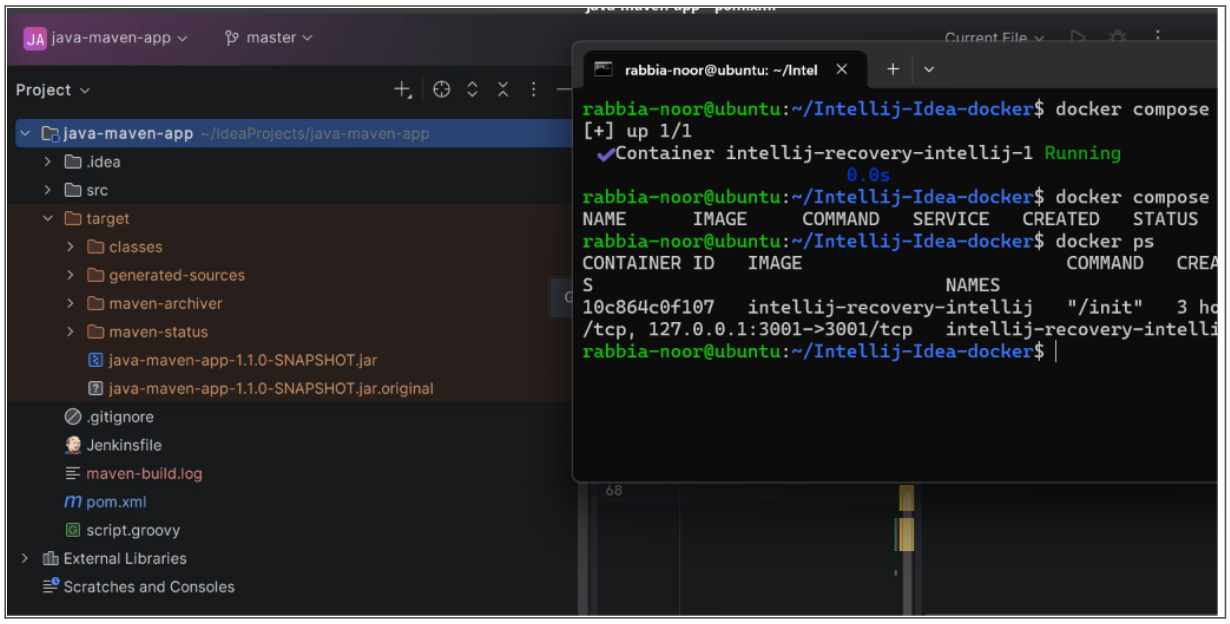
Step 5: Ran mvn package again after adding the dependency. It built target/java-maven-app-1.1.0-SNAPSHOT.jar and finished with BUILD SUCCESS (01:48 min).



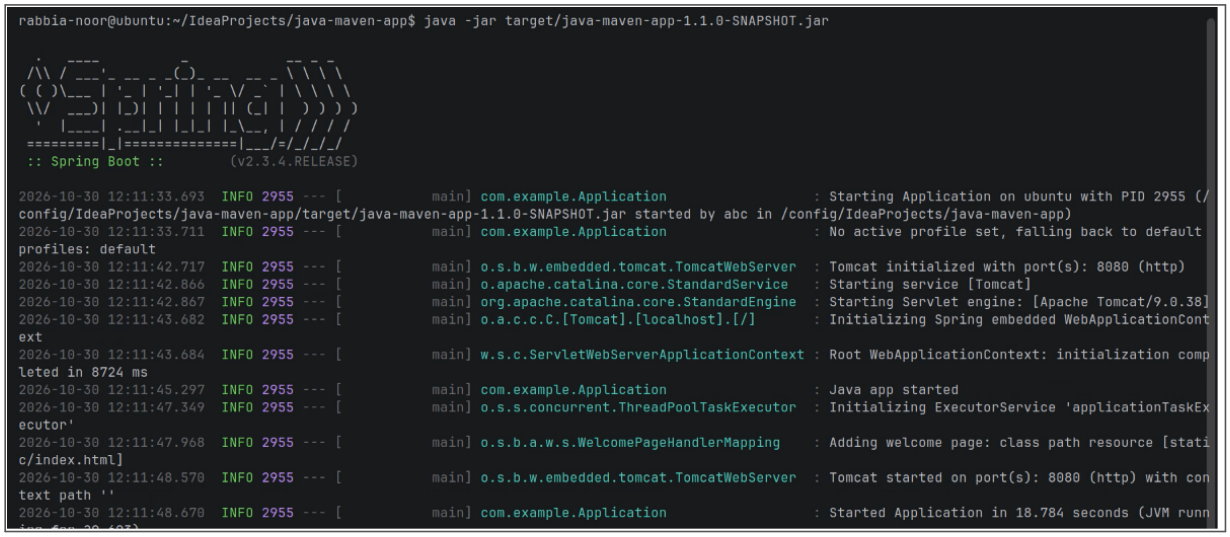
Step 6: Ran mvn test. Surefire reported No tests to run because the sample has no test sources, and the build finished with BUILD SUCCESS (02:28 min).



Step 7: In the IntelliJ Project pane, the target directory contains java-maven-app-1.1.0-SNAPSHOT.jar (and the .jar.original file). The host terminal showing rabbia-noor@ubuntu is on the right.



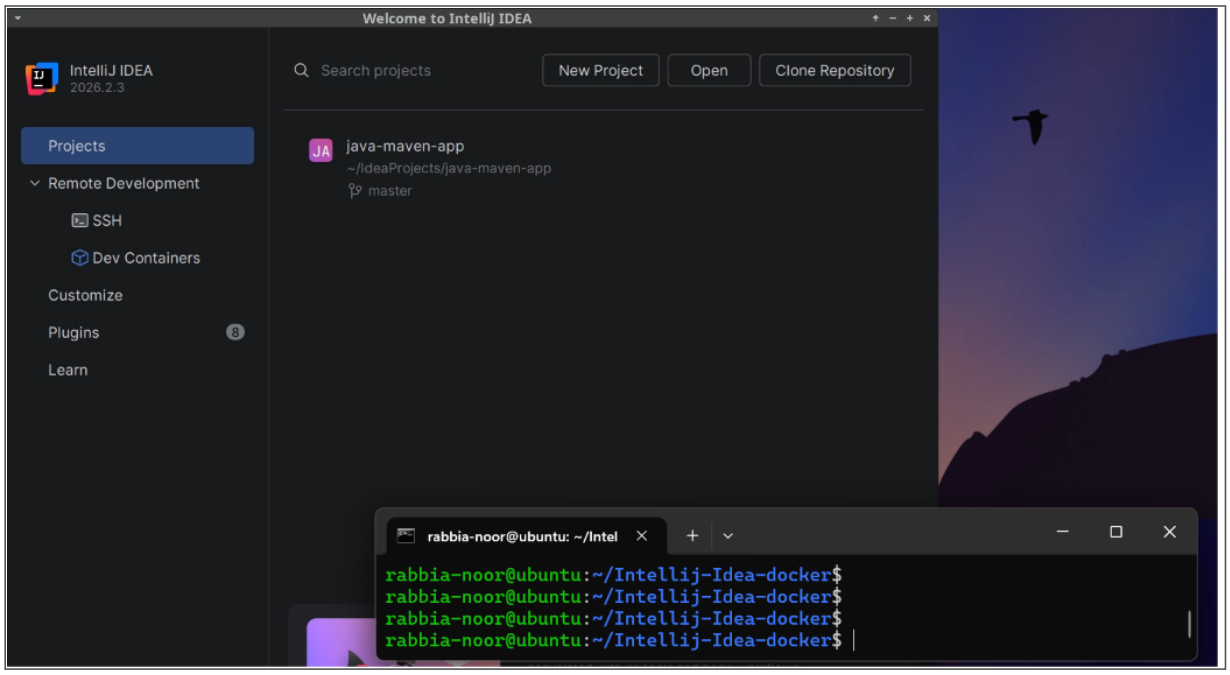
Step 8: Ran java -jar target/java-maven-app-1.1.0-SNAPSHOT.jar. Spring Boot 2.3.4 started Tomcat on port 8080 and logged Started Application in 18.784 seconds.



Step 9: Pressed Ctrl+C. The ^C and the Spring Boot shutdown message appear and the prompt rabbia-noor@ubuntu:~/IdeaProjects/java-maven-app$ returned.

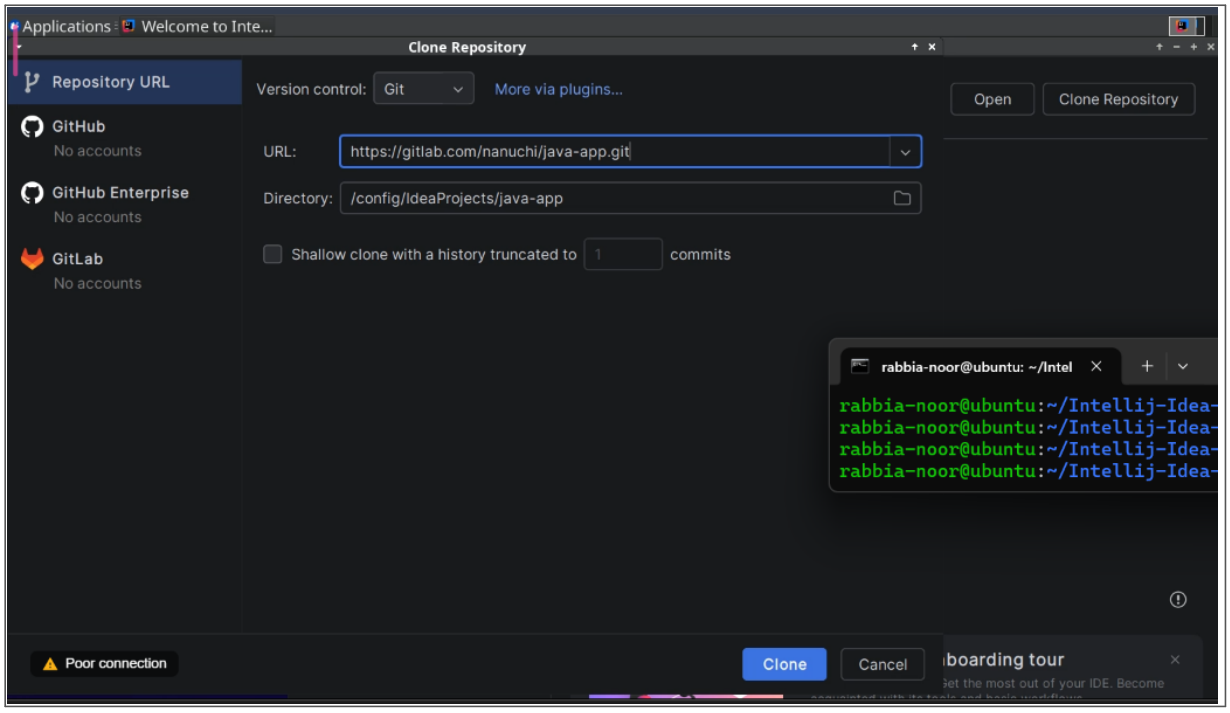


Step 10: Closed the project with File → Close Project. The IntelliJ Welcome screen is shown, with the host terminal (rabbia-noor@ubuntu) beside it.



Task 3 – Build and Run the Gradle Project

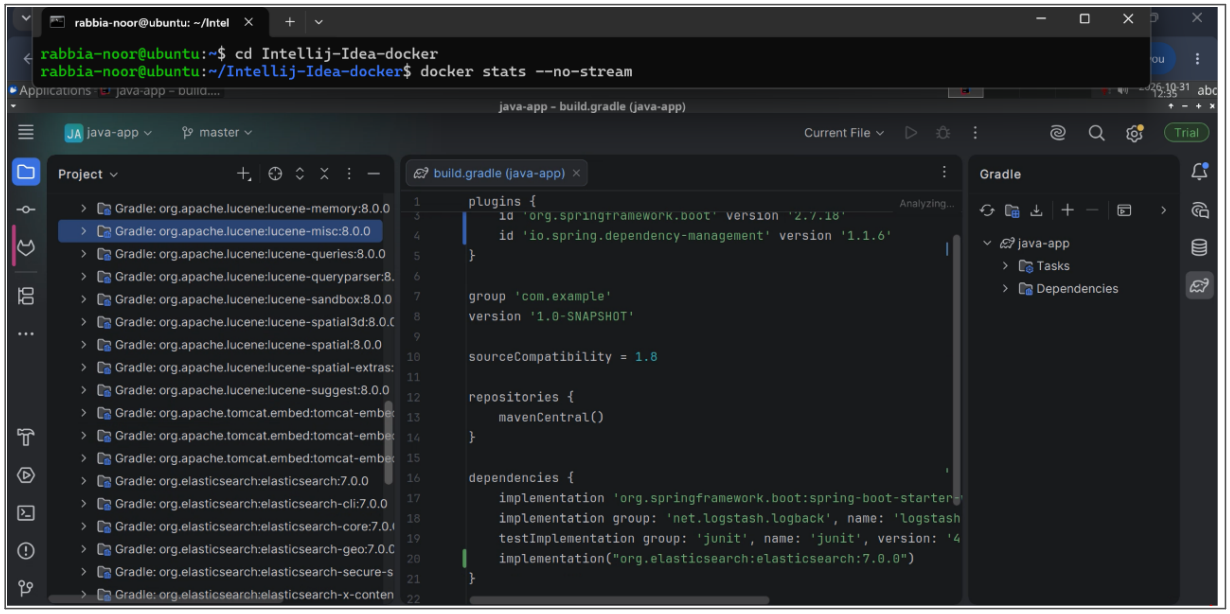
Step 1: Cloned https://gitlab.com/nanuchi/java-app.git into /config/IdeaProjects/java-app using IntelliJ's Clone Repository dialog. The host terminal prompt is visible.



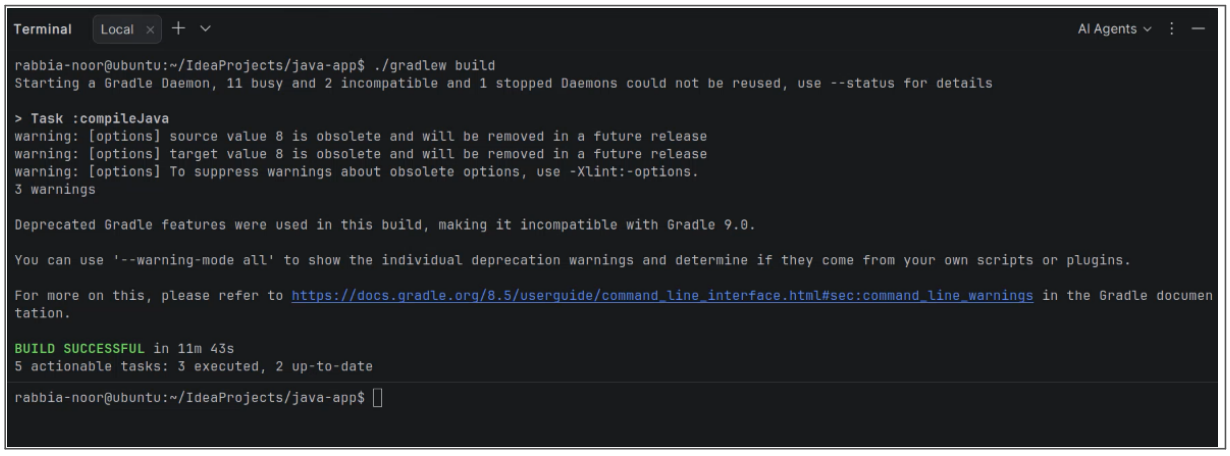
Step 2: Ran ./gradlew build with the Gradle Wrapper from the project root. The build ended with BUILD SUCCESSFUL in 16m 49s (5 actionable tasks: 5 executed).



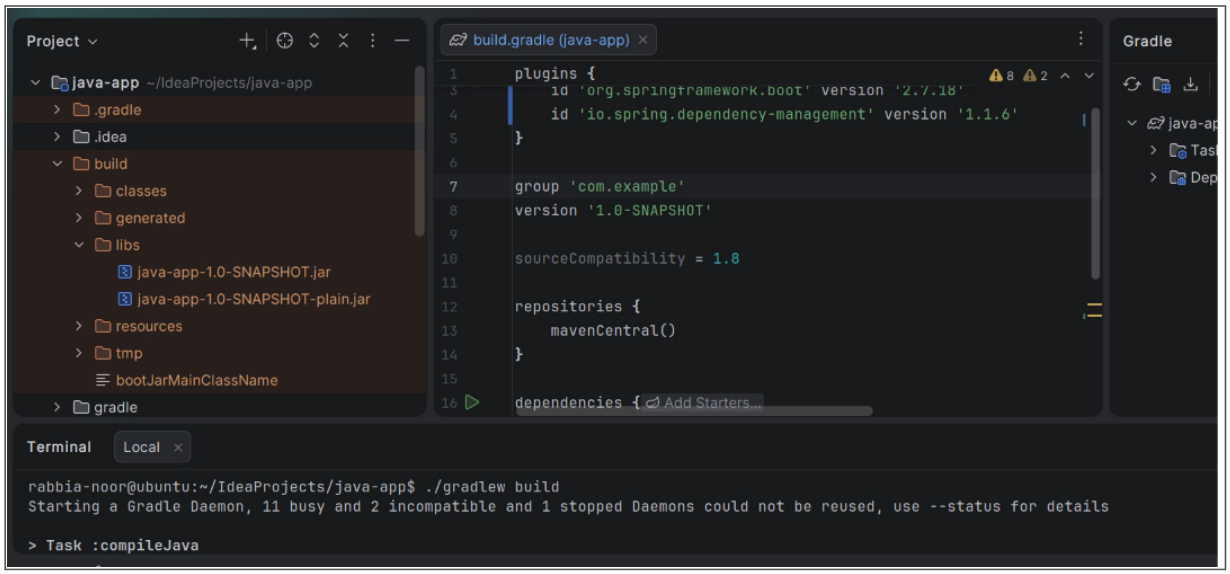
Step 3: Added implementation("org.elasticsearch:elasticsearch:7.0.0") to the dependencies block of build.gradle and synced. External Libraries now lists Gradle: org.elasticsearch:elasticsearch:7.0.0. The host terminal prompt rabbia-noor@ubuntu is visible at the top.



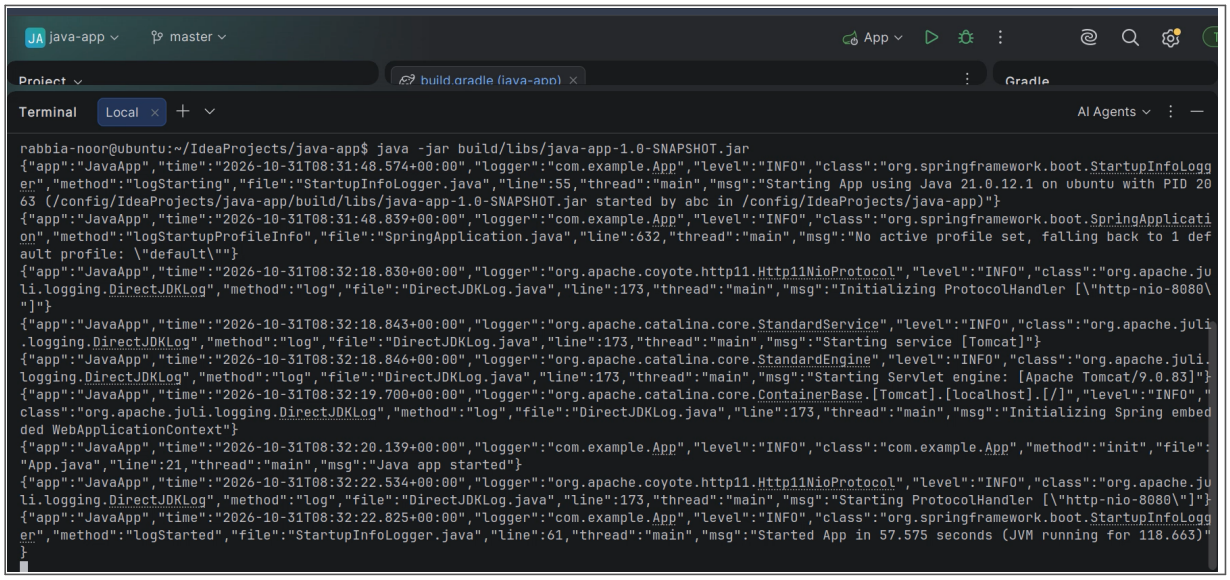
Step 4: Ran ./gradlew build again after the dependency change. It ended with BUILD SUCCESSFUL in 11m 43s (5 actionable tasks: 3 executed, 2 up-to-date).



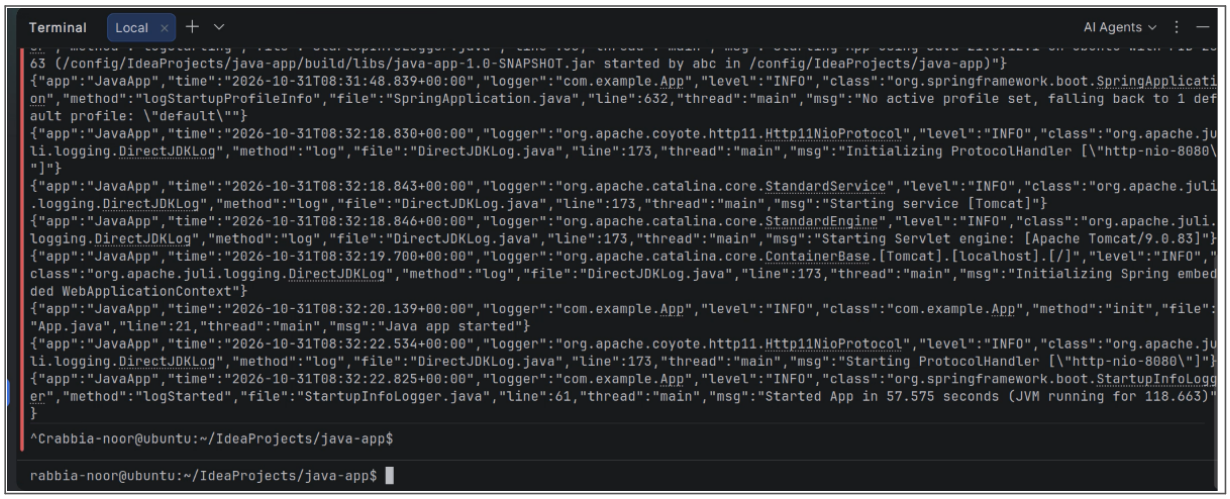
Step 5: In the Project pane, build → libs contains java-app-1.0-SNAPSHOT.jar (and java-app-1.0-SNAPSHOT-plain.jar). The IntelliJ terminal below shows the rabbia-noor@ubuntu prompt.



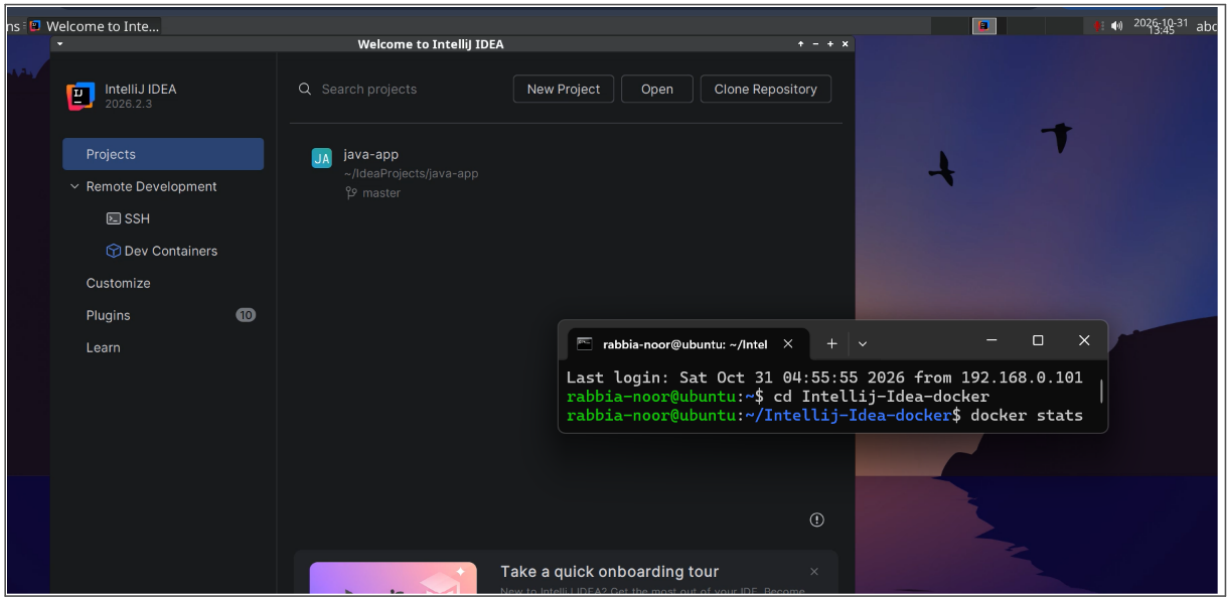
Step 6: Ran java -jar build/libs/java-app-1.0-SNAPSHOT.jar. Tomcat started on port 8080 and the application logged Started App in 57.575 seconds.



Step 7: Pressed Ctrl+C. The application stopped, ^C is shown and the prompt rabbia-noor@ubuntu:~/IdeaProjects/java-app$ returned, which frees port 8080.

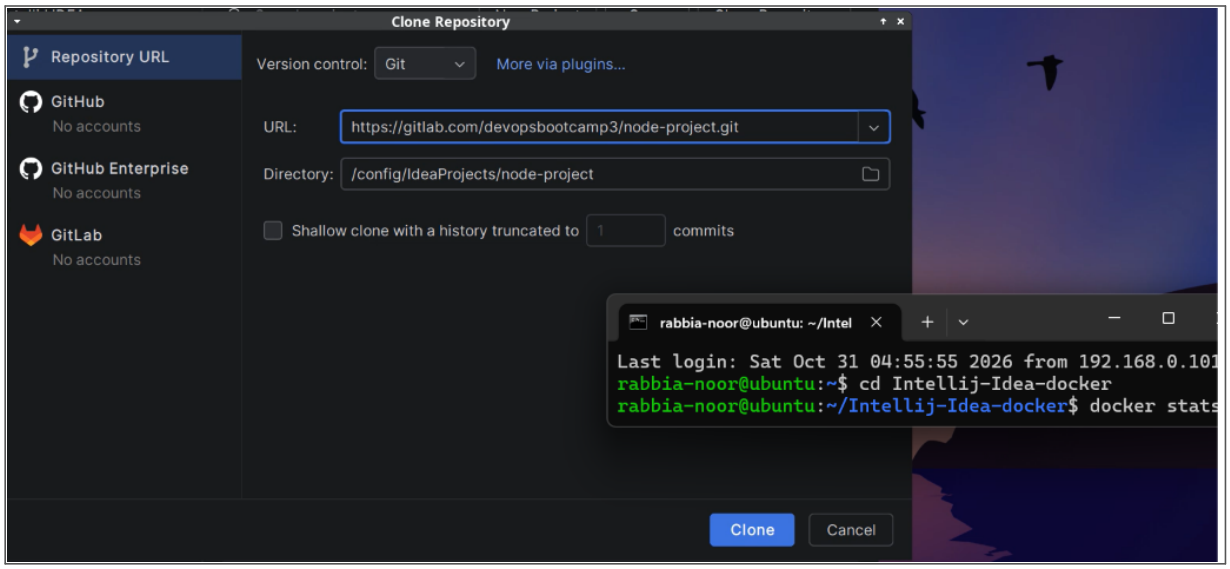


Step 8: Closed the project with File → Close Project. The Welcome screen is shown, with the Ubuntu host terminal (rabbia-noor@ubuntu) beside it.



Task 4 – Install and Run the Node.js Project

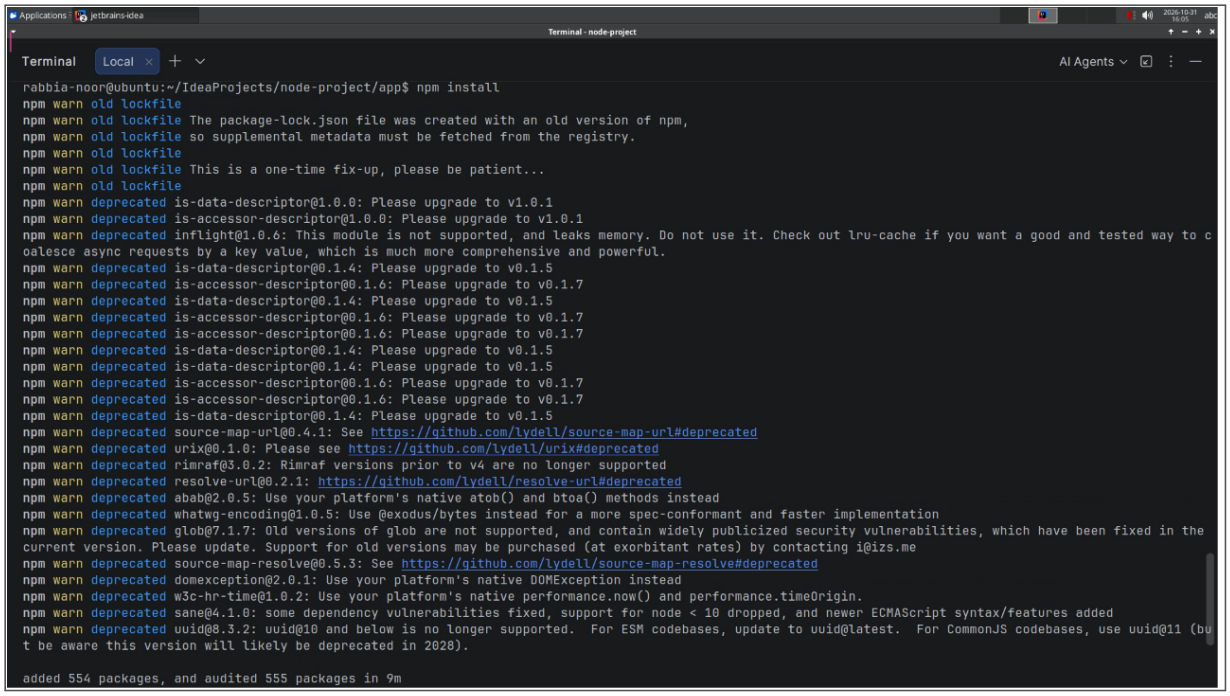
Step 1: Cloned https://gitlab.com/devopsbootcamp3/node-project.git into /config/IdeaProjects/node-project with Clone Repository. The host terminal prompt is visible.



Step 2: Installed Node.js and npm with sudo apt install -y nodejs npm. node --version shows v22.23.4 and npm --version shows 10.9.9.



Step 3: Ran npm install inside the app directory. npm finished with added 554 packages, and audited 555 packages.



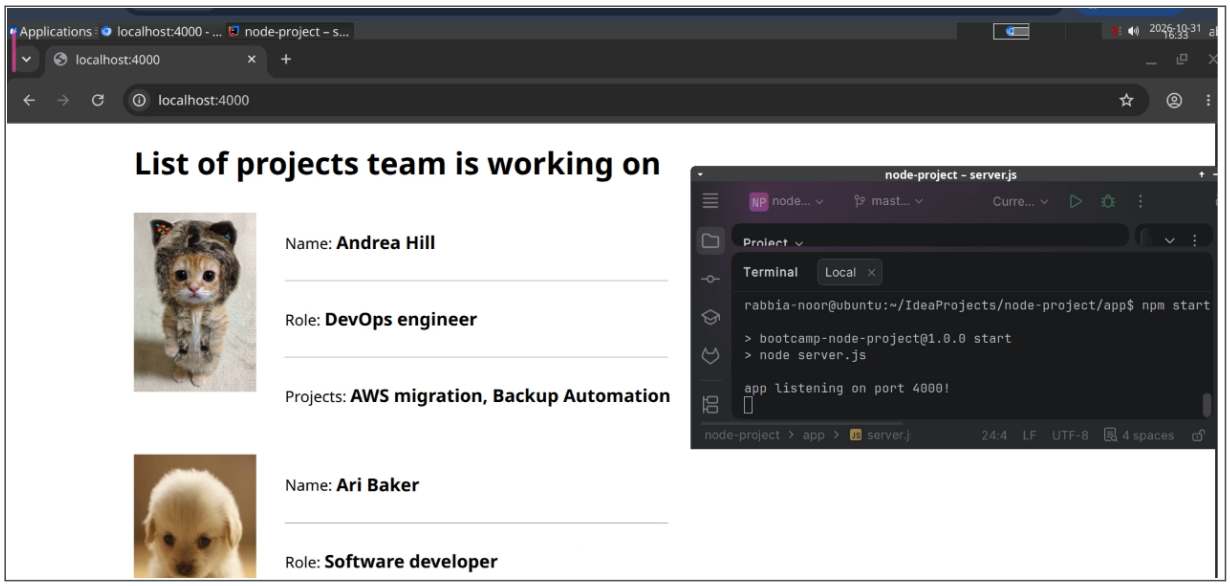
Step 4: Ran npm start after changing the listener in app/server.js from port 3000 to 4000 (port 3000 is used by Webtop in this container). The server prints app Listening on port 4000!.

```
rabbia-noor@ubuntu:~/IdeaProjects/node-project/app$ npm start

> bootcamp-node-project@1.0.0 start
> node server.js

app listening on port 4000!
```

Step 5: Opened http://localhost:4000 in the container desktop's browser. The Node.js project page (team members and their profile images) is visible, with the IntelliJ terminal showing npm start (rabbia-noor@ubuntu) in a split view.



Step 6: Stopped the app with Ctrl+C and ran npm test. Jest passed: main index.html file exists, Test Suites: 1 passed, Tests: 1 passed.

```
rabbia-noor@ubuntu:~/IdeaProjects/node-project/app$ npm test

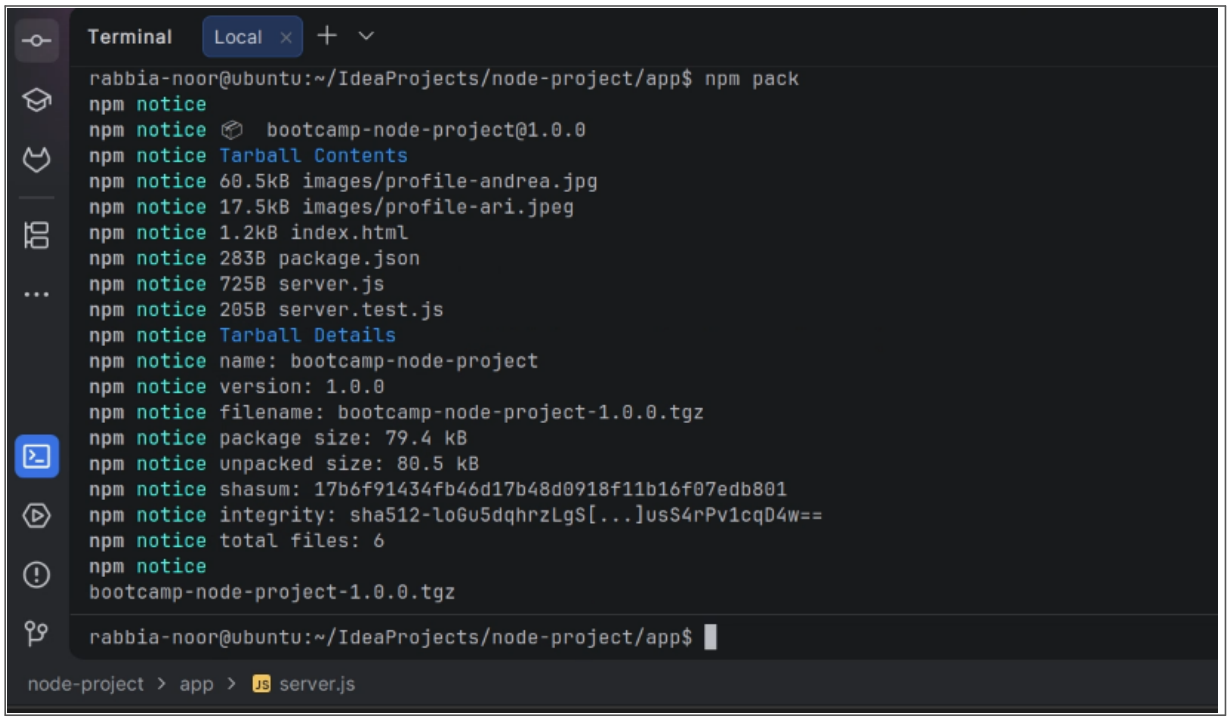
> bootcamp-node-project@1.0.0 test
> jest

Browserslist: caniuse-lite is outdated. Please run:
npx browserslist@latest --update-db

Why you should do it regularly:
https://github.com/browserslist/browserslist#browsers-data-updating
PASS ./server.test.js (10.775 s)
  ✓ main index.html file exists (6 ms)

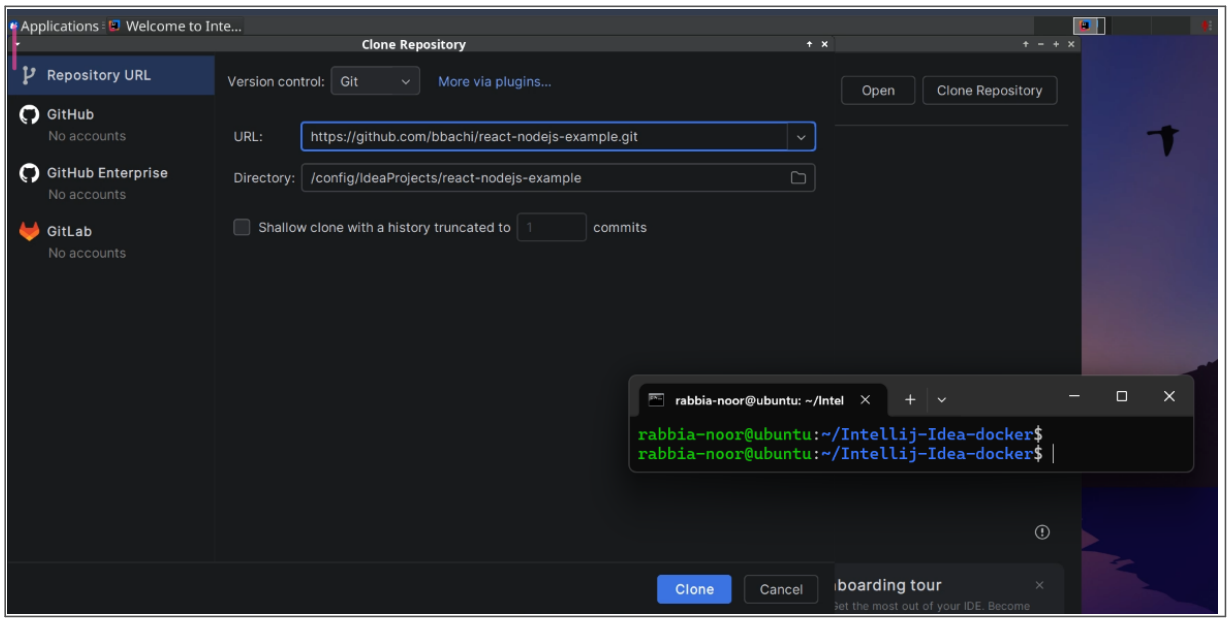
Test Suites: 1 passed, 1 total
Tests:       1 passed, 1 total
Snapshots:   0 total
Time:        45.648 s
Ran all test suites.
```

Step 7: Ran npm pack in the app directory. npm listed the six packaged files and created bootcamp-node-project-1.0.0.tgz (package size 79.4 kB).

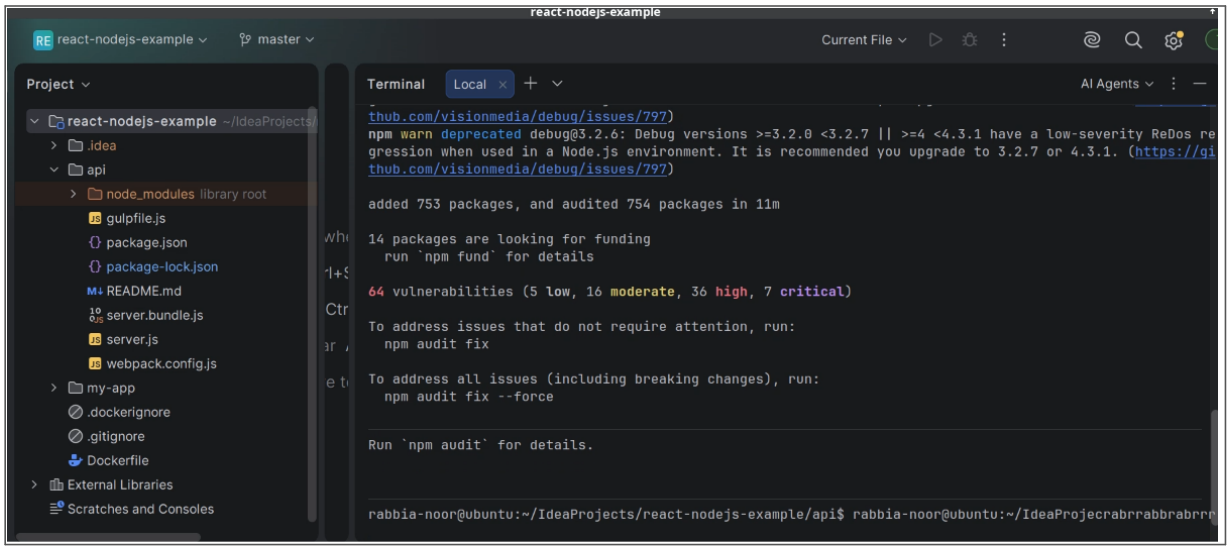


Task 5 – Build the React and Node.js API Artifact

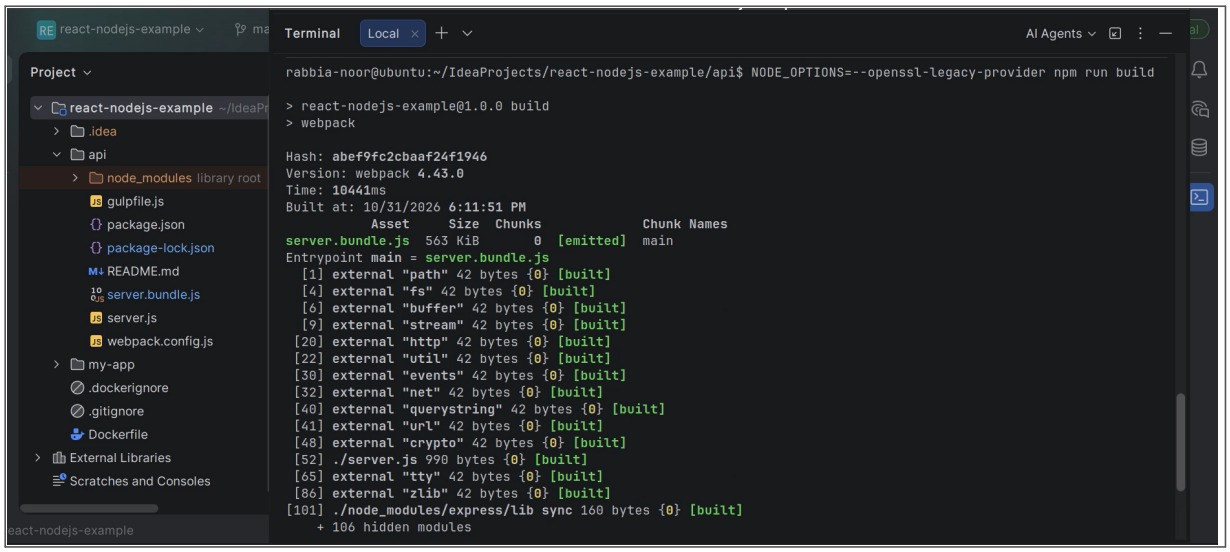
Step 1: Cloned https://github.com/bbachi/react-nodejs-example.git into /config/IdeaProjects/react-nodejs-example. The host terminal prompt is visible in the screenshot.



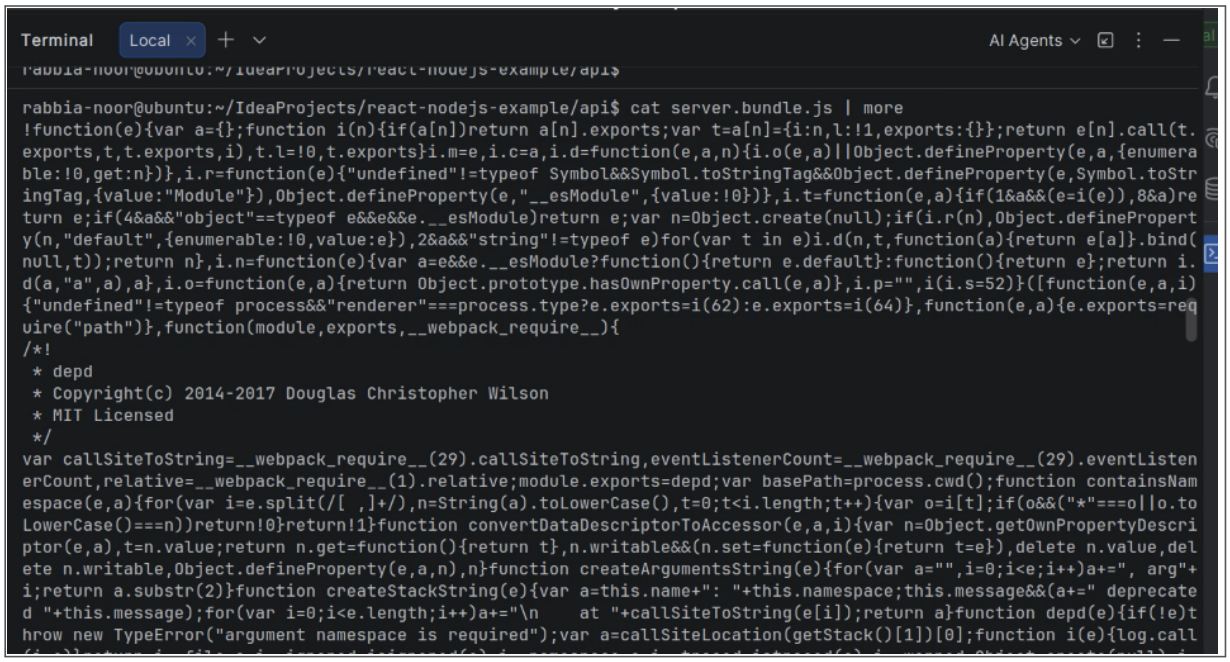
Step 2: Ran npm install inside the api directory. npm finished with added 753 packages, and audited 754 packages, and api/node_modules is shown in the Project pane.



Step 3: Ran NODE_OPTIONS=--openssl-legacy-provider npm run build in the api directory (the legacy option is needed for this older webpack 4.43.0 on the current Node.js). webpack emitted server.bundle.js (563 KiB), which is listed in the api directory.

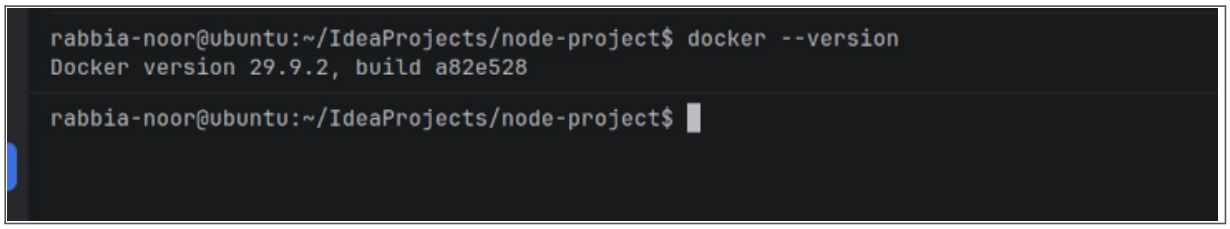


Step 4: Viewed the generated bundle with cat server.bundle.js | more. The minified webpack output is displayed in the IntelliJ terminal (rabbia-noor@ubuntu).

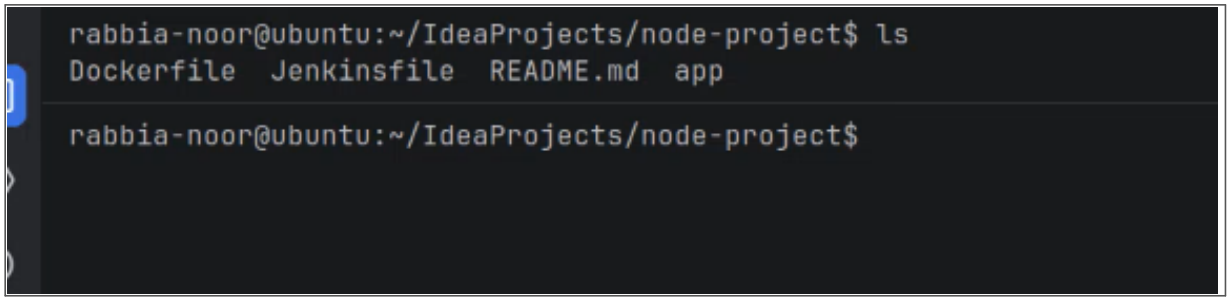


Task 6 – Build a Docker Image for the Node.js Project

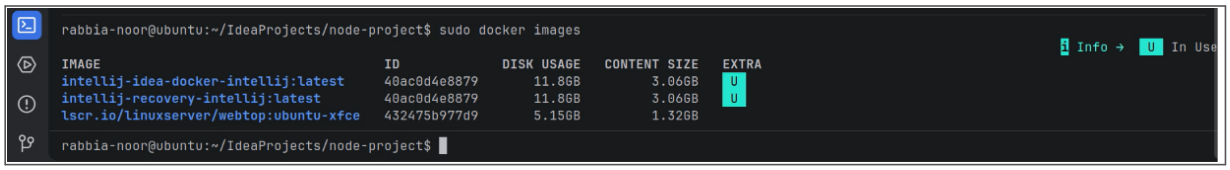
Step 1: In the node-project root, docker --version shows Docker version 29.9.2.



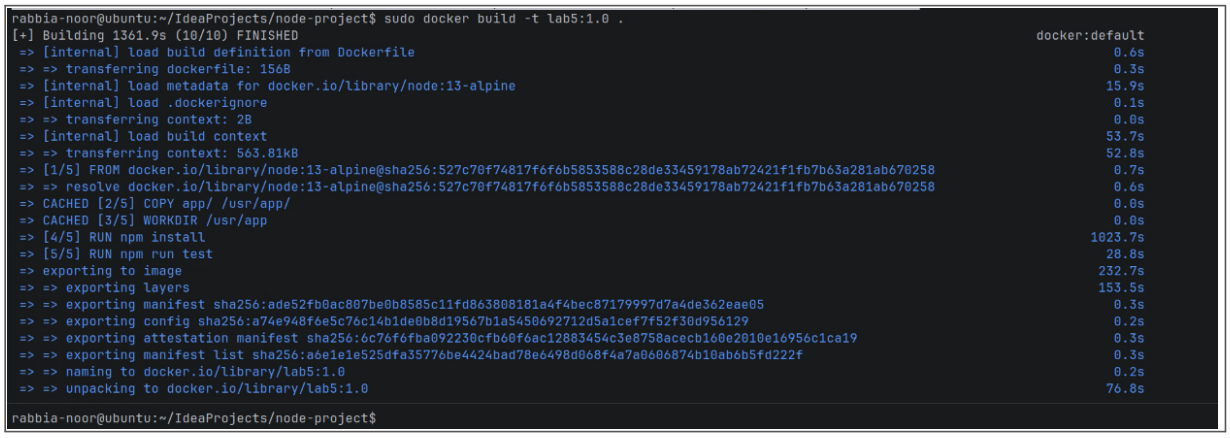
Step 2: ls in /config/IdeaProjects/node-project lists Dockerfile, Jenkinsfile, README.md and app.



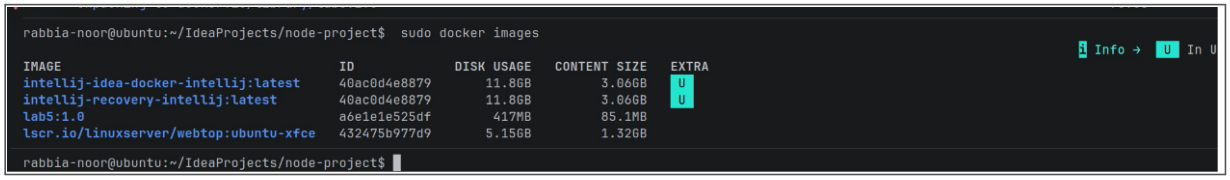
Step 3: sudo docker images before the build lists intellij-idea-docker-intellij, intellij-recovery-intellij and the lscr.io/linuxserver/webtop image, and no lab5 image yet.



Step 4: Ran sudo docker build -t lab5:1.0 . from the repository root. Docker copied app/, ran npm install and npm run test, and finished with naming to docker.io/library/lab5:1.0.



Step 5: sudo docker images now lists the new image: repository lab5, tag 1.0, image ID a6e1e1e525df (417 MB), together with the earlier images.



Build and Run Results

Result of each build, test and run, taken from the screenshots above.

Project	Command	Result	Artifact / output
Maven	mvn package (before and after Elasticsearch 7.0.0)	BUILD SUCCESS	target/java-maven-app-1.1.0-SNAPSHOT.jar
Maven	mvn test	BUILD SUCCESS (no tests to run)	No test sources in the sample
Maven	java -jar target/...jar	Started Application, Tomcat on 8080	Stopped with Ctrl+C
Gradle	./gradlew build (before and after Elasticsearch 7.0.0)	BUILD SUCCESSFUL	build/libs/java-app-1.0-SNAPSHOT.jar
Gradle	java -jar build/libs/...jar	Started App, Tomcat on 8080	Stopped with Ctrl+C
Node.js	npm install / npm start	554 packages, port 4000	http://localhost:4000 page
Node.js	npm test	1 test suite, 1 test passed	Jest: index.html exists
Node.js	npm pack	Tarball created	bootcamp-node-project-1.0.0.tgz
React/Node API	npm run build (webpack 4.43.0)	server.bundle.js emitted (563 KiB)	api/server.bundle.js
Docker	sudo docker build -t lab5:1.0 .	Image built and tagged	lab5:1.0 (ID a6e1e1e525df)

Exam Evaluation Questions

Exam Question 1 – How is the Maven JAR name decided, and what does the package phase do?

Answer: The name comes from the values in pom.xml: the **artifactId** (java-maven-app) and the **version** (1.1.0-SNAPSHOT), joined as artifactId-version with the extension from the **packaging** type (jar). That gives java-maven-app-1.1.0-SNAPSHOT.jar in the target directory.

The **package** phase is part of Maven's default lifecycle. Running mvn package runs every earlier phase first (validate, compile, test), then packages the compiled classes and resources into the JAR. In this project the Spring Boot Maven plugin also repackages that JAR into an executable one (the output shows "Replacing main artifact with repackaged archive"), and the original is kept as java-maven-app-1.1.0-SNAPSHOT.jar.original.

Exam Question 2 – Why does ./gradlew build work without a system-wide Gradle installation?

Answer: The project contains the Gradle Wrapper (gradlew, gradlew.bat and gradle/wrapper/gradle-wrapper.properties). The properties file names the Gradle version to use, and on the first run the script downloads that version (8.5 in the build output) into the user's Gradle home and runs it, so only a JDK is needed.

If the sample reports an unsupported Java version, I would check: the **Project SDK** (File → Project Structure), the **Gradle JVM** (Settings → Build, Execution, Deployment → Build Tools → Gradle), the JAVA_HOME and java -version used by the terminal, and the sourceCompatibility value in build.gradle (1.8 here). The Gradle version in the wrapper must also support the installed JDK (the Java compatibility table in the Gradle documentation).

Exam Question 3 – Why does EADDRINUSE occur on port 3000, and why does port 4000 fix it?

Answer: EADDRINUSE means that another process is already listening on the port. In this environment the Webtop desktop inside the container already uses port 3000, so server.js cannot bind to it and Node.js fails with EADDRINUSE.

Changing app.listen(3000, ...) to app.listen(4000, ...) makes the application bind to a free port. The browser of the container desktop runs in the same container as IntelliJ and Node.js, so http://localhost:4000 reaches the application directly and the page opens.

DevOps repository (Labs/Lab05):

https://github.com/Rabbia-Noor/DevOps